

PAGE 1

VERIQTA
ENGINEERING EXCELLENCE

AWS SECURITY
HANDBOOK

BUILD SECURE.
RUN SECURE.
SCALE SECURE.

✓ PROTECT
✓ DETECT
✓ RESPOND
✓ RECOVER



CLOUD
SECURITY
DONE RIGHT

AWS SECURITY HANDBOOK

FROM CLOUD SECURITY FOUNDATIONS
TO PRODUCTION DEFENSE

IDENTITY
ACCESS
SECURITY

DATA
PROTECTION

NETWORK
DEFENSE

THREAT
DETECTION
& RESPONSE



SECURE
IDENTITIES



PROTECT
INFRASTRUCTURE



SECURE
DATA



DETECT &
RESPOND



COMPLY &
GOVERN



BEST PRACTICES
REAL-WORLD
SECURITY GUIDANCE



REAL-WORLD
SCENARIOS
PRACTICAL EXAMPLES
FROM PRODUCTION



HANDS-ON
APPROACH
ACTIONABLE STEPS
YOU CAN IMPLEMENT



PRODUCTION
READY
SECURE, MONITOR,
AND SCALE
WITH CONFIDENCE

SECURITY

COMPLIANCE

RESILIENCE

VISIBILITY

INNOVATION

WE BUILD
SECURE CLOUDS

VERIQTA
ENGINEERING EXCELLENCE



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

BETTER
SYSTEMS.
A SAFER
TOMORROW.



AWS SECURITY FOUNDATIONS

① SHARED RESPONSIBILITY MODEL

AWS RESPONSIBILITY (SECURITY OF THE CLOUD)	YOUR RESPONSIBILITY (SECURITY IN THE CLOUD)
- GLOBAL INFRASTRUCTURE - REGIONS & AZs - PHYSICAL SECURITY - HARDWARE - NETWORKING - VIRTUALIZATION - AWS SERVICES	- DATA - IAM: USERS & ROLES - DEVICE & ENDPOINTS - APPLICATIONS - NETWORK CONFIGURATION - OPERATING SYSTEM - FIREWALL & SECURITY GROUPS

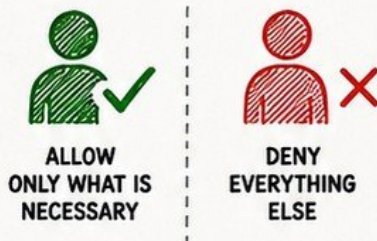
② DEFENSE IN DEPTH



MULTIPLE LAYERS.
MULTIPLE CONTROLS.
REDUCE RISK.

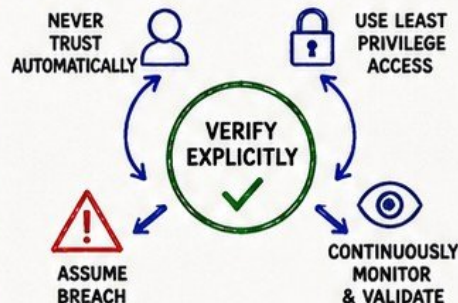
- PERIMETER SECURITY (WAF, SHIELD, FIREWALL)
- NETWORK SECURITY (VPC, SG, NACL)
- IDENTITY SECURITY (IAM, MFA, ROLES)
- WORKLOAD SECURITY (PATCHING, ANTIVIRUS)
- DATA SECURITY (ENCRYPTION, KMS)
- MONITORING & DETECTION (GUARDDUTY, CLOUDTRAIL)

③ LEAST PRIVILEGE



LIMIT ACCESS.
REDUCE RISK.
IMPROVE SECURITY.

④ ZERO TRUST PRINCIPLES



TRUST NO ONE.
VERIFY EVERYONE.
SECURE EVERYTHING.

⑤ AWS WELL-ARCHITECTED SECURITY PILLAR



- STRONG IDENTITY FOUNDATION
- TRACEABILITY
- INFRASTRUCTURE PROTECTION
- DATA PROTECTION
- SECURITY IN ALL LAYERS
- AUTOMATION & CONTINUOUS IMPROVEMENT

BUILD SECURE.
OPERATE SECURE.
EVOLVE SECURE.

⑥ SECURITY RESPONSIBILITIES IN PRODUCTION

IDENTITY & ACCESS MANAGEMENT

- IAM ROLES
- LEAST PRIVILEGE
- MFA ENABLEMENT

PROTECT INFRASTRUCTURE

- SECURE NETWORKING
- PATCH & HARDEN
- MONITOR CONTINUOUSLY

PROTECT DATA

- ENCRYPT AT REST
- ENCRYPT IN TRANSIT
- MANAGE KEYS SECURELY

DETECT & RESPOND

- CENTRALIZED LOGGING
- THREAT DETECTION
- INCIDENT RESPONSE

COMPLY & IMPROVE

- MONITOR COMPLIANCE
- AUTOMATE CONTROLS
- REVIEW & IMPROVE

SECURITY IS EVERYONE'S JOB.
BUILD IT. RUN IT. OWN IT.

VERIQTA
ENGINEERING EXCELLENCE



SECURE TODAY.
CONFIDENT TOMORROW.



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta



SECURING THE AWS ROOT USER AND ACCOUNT



1 ROOT-USER RISKS



- ✓ Full account control
- ✓ Accidental deletion
- ✓ Security misconfiguration
- ✓ Human error

2 MFA (MULTI-FACTOR AUTHENTICATION)



- ✓ Enable MFA on root user
- ✓ Use authenticator app
- ✓ Avoid SMS only where possible
- ✓ Keep MFA active

3 ROOT CREDENTIALS



- ✓ Never share root keys
- ✓ Store credentials securely
- ✓ Rotate if compromised
- ✓ Limit root user usage

4 ACCOUNT CONTACTS



- ✓ Keep root and billing contacts up to date
- ✓ Use trusted email and phone
- ✓ Verify contact ownership
- ✓ Update when changes occur

5 ALTERNATE SECURITY CONTACTS



- ✓ Add secondary contacts
- ✓ Ensure multiple trusted people
- ✓ Keep information current
- ✓ Avoid single point of failure

6 ROOT-USER EMERGENCY PROCEDURES



- ✓ Disable root access if compromised
- ✓ Rotate credentials
- ✓ Review account activity
- ✓ Contact AWS Support

7 ACCOUNT SECURITY BASELINE



- ✓ Enable MFA on root user
- ✓ Create IAM users for daily work
- ✓ Apply least privilege
- ✓ Keep contacts up to date
- ✓ Monitor account activity



KEY GOAL:

Keep the AWS root user secure and unused for daily operations.



SECURE THE
ROOT USER.



PROTECT
YOUR AWS ACCOUNT.



LESS RISK.
MORE SECURITY.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta



IAM FUNDAMENTALS



AWS IDENTITY AND ACCESS MANAGEMENT (IAM)
LET YOU SECURELY CONTROL WHO CAN ACCESS
AWS RESOURCES AND WHAT THEY CAN DO.

1 IAM USERS



Represents an individual who can access AWS.

- ✓ Console or programmatic access
- ✓ Has unique credentials
- ✓ Assigned permissions directly or via groups

2 IAM GROUPS



A collection of users. Makes permissions management easier.

- ✓ Attach policies to groups
- ✓ Add users to groups
- ✓ Users inherit group permissions

3 IAM ROLES



An identity that AWS services or trusted entities can assume.

- ✓ Used by AWS services, applications or users
- ✓ Temporary credentials
- ✓ Cross-account access
- ✓ No long-term credentials

4 IAM POLICIES



JSON documents that define permissions.

- ✓ Define what is allowed or denied
- ✓ Attach to users, groups or roles
- ✓ Enable least privilege

5 IDENTITY-BASED POLICIES



Attached to an identity (user, group, role). These policies control what the identity can do.

EXAMPLE

```
{
  "Effect": "Allow",
  "Action": "s3:ListBucket",
  "Resource": "arn:aws:s3:::my-bucket"
}
```

6 RESOURCE-BASED POLICIES



Attached to a resource (like S3 bucket, SQS queue). These policies control WHO can access the resource.

EXAMPLE

```
{
  "Effect": "Allow",
  "Principal": { "AWS": "arn:aws:iam::123456789012:role/AppRole" },
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3:::my-bucket/*"
}
```

7 AUTHENTICATION vs AUTHORIZATION



AUTHENTICATION

Proves WHO you are.

- ✓ Verifies identity
- ✓ Examples: Password, MFA, Access keys, SSO
- ✓ Answers: "Are you who you say you are?"

VS

AUTHORIZATION

Determines WHAT you can do.

- ✓ Checks permissions
- ✓ Based on policies
- ✓ Answers: "What are you allowed to do?"



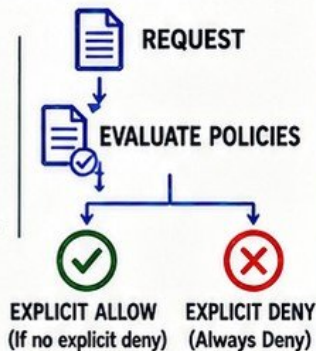
IAM IS THE FOUNDATION OF AWS SECURITY.
USE LEAST PRIVILEGE. REVIEW REGULARLY. STAY SECURE.



ADVANCED IAM AND LEAST PRIVILEGE

1 POLICY EVALUATION LOGIC

AWS evaluates all applicable policies and determines if the request is **ALLOWED** or **DENIED**.



EXPLICIT DENY ALWAYS OVERRIDES ALLOW

2 EXPLICIT ALLOW AND DENY



EXPLICIT ALLOW

- Grants permission
- Allows the action if no explicit deny exists



EXPLICIT DENY

- Overrides everything
- Denies the action even if an allow exists elsewhere

3 PERMISSION BOUNDARIES



Sets the maximum permissions an identity can have.

- Does not grant permissions
- Limits permissions from identity policies
- Helps enforce least privilege
- Prevents privilege escalation

4 CONDITIONS



Add logic to your policies using condition keys.

Examples:

- aws:SourceIp
- aws:MultiFactorAuthPresent
- s3:prefix
- aws:CurrentTime

CONDITIONS = CONTEXT-AWARE ACCESS

5 TEMPORARY CREDENTIALS



- Short-lived credentials
- Automatically expire
- Reduce the risk of credential compromise
- Used by roles, federation and STS

SHORT-LIVED IS MORE SECURE

6 ROLE ASSUMPTION



- Assume a role to get permissions
- Uses STS (Security Token Service)
- No long-term credentials
- Supports cross-account and federation

USE ROLES, NOT USERS

7 CROSS-ACCOUNT ACCESS



- Access resources in another AWS account
- Use roles + trust policies
- Common for centralized teams and services
- Improves security and isolation

ACCESS ACROSS ACCOUNTS SECURELY

8 IAM ACCESS ANALYZER



Helps you identify and analyze public and cross-account access to your AWS resources.

- Finds resources shared externally
- Identifies unused access
- Checks policies for external access
- Validates least privilege
- Supports continuous monitoring

COMMON FINDINGS



Public access to your resources



Cross-account access not expected



Policies that allow more than required



**LEAST PRIVILEGE IS NOT JUST A BEST PRACTICE.
IT IS YOUR FIRST LINE OF DEFENSE.**



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta



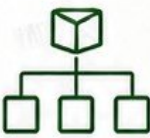
AWS ORGANIZATIONS AND MULTI-ACCOUNT SECURITY

1 AWS ORGANIZATIONS



Centrally manage multiple AWS accounts. Set guardrails, enforce policies, and automate governance.

2 ORGANIZATIONAL UNITS (OU)



Group related accounts. Apply policies to OUs instead of individual accounts.

3 SERVICE CONTROL POLICIES (SCPs)



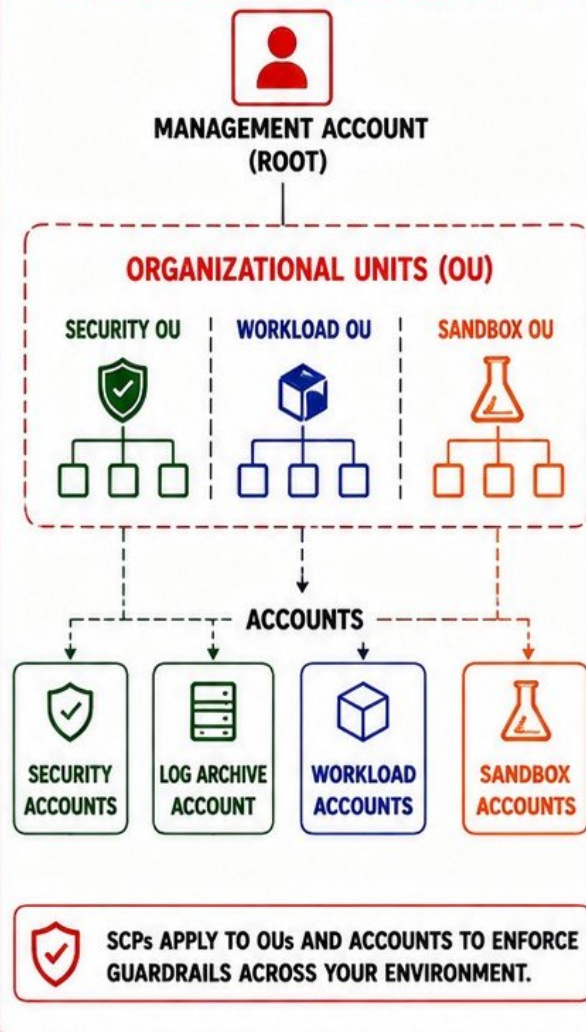
Set maximum permissions for accounts in an OU. They act as guardrails and prevent unwanted actions.

4 MANAGEMENT ACCOUNT



The master account that creates and manages the organization. Do not use it for daily workloads.

AWS ORGANIZATIONS STRUCTURE



5 SECURITY ACCOUNTS



Dedicated accounts for security services like GuardDuty, Security Hub, IAM Identity Center, and Inspector.

6 LOG ARCHIVE ACCOUNT



Centralized account to store and protect logs from CloudTrail, Config, VPC Flow Logs, etc.

7 WORKLOAD SEPARATION



Isolate workloads by environment, team, or application. Reduce blast radius and improve security.

8 DELEGATED ADMINISTRATION



Delegate AWS service management to specific accounts instead of using the management account.

WHY MULTI-ACCOUNT STRATEGY MATTERS



STRONGER SECURITY
Isolates workloads and limits blast radius.



COST MANAGEMENT
Track costs by team, project, or environment.



COMPLIANCE AND AUDIT
Centralized logging and clear account separation.



OPERATIONAL EFFICIENCY
Teams get the access they need without risk.



GOVERNANCE AT SCALE
Enforce policies consistently with SCPs.



BUILD A SECURE FOUNDATION. SEPARATE, GOVERN, MONITOR, AND PROTECT. THAT IS THE POWER OF AWS ORGANIZATIONS.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

NETWORK SECURITY FUNDAMENTALS

BUILD SECURE, ISOLATED, AND CONTROLLED NETWORKS IN AWS

1 VPC SECURITY BOUNDARIES



- Your VPC is logically isolated from other AWS accounts.
- Control traffic in and out of your network.
- Use security groups, NACLs, route tables, and gateways to enforce boundaries.

2 PUBLIC VS PRIVATE SUBNETS



PUBLIC SUBNET

- Has route to Internet Gateway.
- Used for resources that must be internet-facing.



PRIVATE SUBNET

- No direct route to Internet.
- Used for backend resources and databases.

3 SECURITY GROUPS



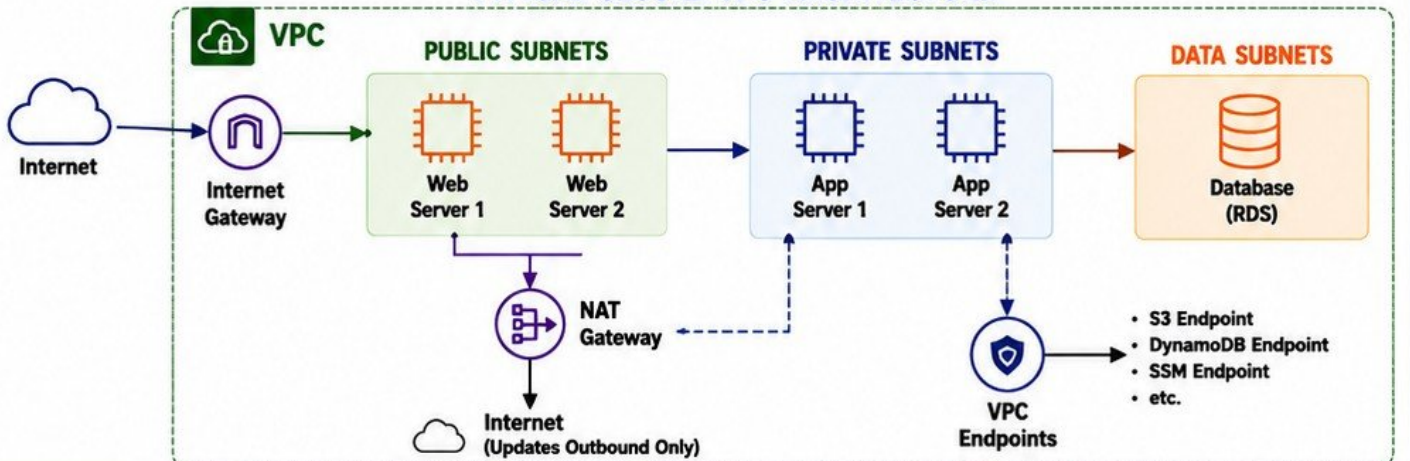
- Stateful firewall at the instance level.
- Allow rules control inbound traffic.
- Allow rules control outbound traffic.
- Only allow what is required.

4 NETWORK ACLS



- Stateless firewall at the subnet level.
- Rules are numbered and evaluated in order.
- Allow or deny inbound and outbound traffic.

TYPICAL SECURE VPC ARCHITECTURE



5 ROUTING SECURITY



- Use route tables to control traffic flow.
- Never route sensitive resources to the Internet.
- Use more specific routes to override broad routes.

6 INTERNET GATEWAY EXPOSURE



- Attach IGW only to public subnets.
- Do NOT attach IGW to private subnets.
- Avoid unnecessary public IPs.

7 NAT GATEWAY CONSIDERATIONS



- Enables outbound internet for private subnets.
- Place in a public subnet.
- Use one NAT GW per AZ for high availability.
- Monitor costs - NAT GW is billed.

8 VPC ENDPOINTS



- Private connection to AWS services.
- Keep traffic within the AWS network.
- Improves security and reduces Data transfer costs.
- Gateway & Interface Endpoints.



KEY TAKEAWAY SECURE YOUR NETWORK AT EVERY LAYER:

SUBNET - FIREWALL - ROUTE - GATEWAY

BEST PRACTICES

- ✓ Implement least privilege for network access.
- ✓ Use private subnets for sensitive resources.
- ✓ Use security groups AND NACLs together.
- ✓ Enable VPC Flow Logs for visibility.

COMMON MISTAKES

- ✗ Exposing databases to the Internet.
- ✗ Using 0.0.0.0/0 in security groups.
- ✗ Not using NAT Gateway in private subnets.
- ✗ No VPC endpoints for AWS services.

ADVANCED NETWORK AND PERIMETER SECURITY

BUILD A STRONG, LAYERED DEFENSE AROUND YOUR AWS ENVIRONMENT

1 AWS WAF



- Protects web applications from common exploits.
- SQLi, XSS, LFI, bots, bad actors.
- Custom rules and managed rule groups.
- Integrated with ALB, CloudFront, API Gateway.

2 AWS SHIELD



- DDoS protection for your applications.
- Shield Standard (automatic).
- Shield Advanced (24/7 DDoS response team).
- Always-on at the network layer.

3 AWS NETWORK FIREWALL



- Stateful, managed network firewall.
- Deep packet inspection.
- IDS/IPS, domain filtering, TLS inspection.
- Scales with your VPC traffic.

4 FIREWALL MANAGER



- Centrally configure and manage firewall rules across accounts.
- Enforce consistent security policies.
- Works with WAF, Network Firewall, Shield Advanced.

5 ROUTE 53 RESOLVER DNS FIREWALL



- Filter DNS queries at the network layer.
- Block access to malicious domains.
- Threat intelligence and domain lists.
- Protects all VPCs using Route 53 Resolver.

6 PRIVATE CONNECTIVITY



- Keep traffic off the public Internet.
- Use VPN, Direct Connect, AWS PrivateLink.
- S2S VPN for hybrid connectivity.
- PrivateLink for AWS services and SaaS.

7 TRAFFIC INSPECTION



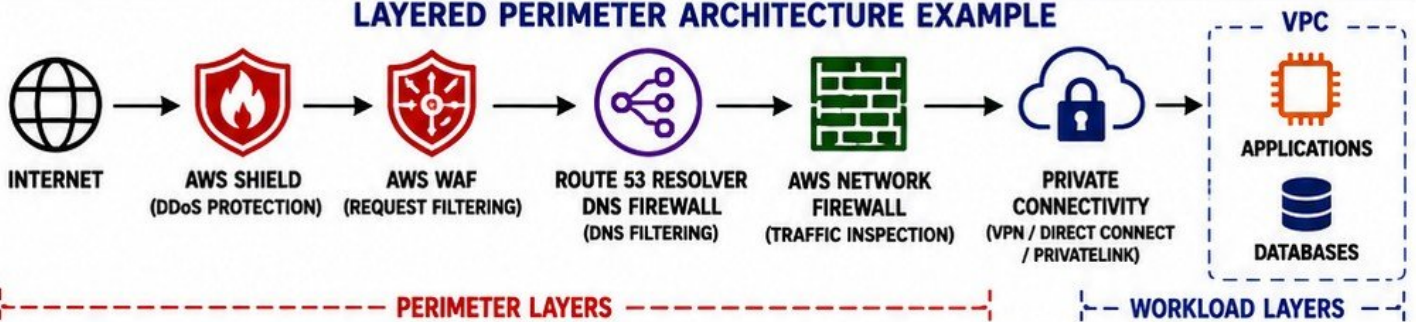
- Inspect traffic for threats and policy enforcement.
- Use Network Firewall for L3-L7 inspection.
- TLS inspection for encrypted traffic.
- Central egress inspection.

8 LAYERED PERIMETER ARCHITECTURE



- Multiple layers of defense.
- Prevent, detect, and respond.
- Reduces blast radius.
- Stronger security posture.

LAYERED PERIMETER ARCHITECTURE EXAMPLE



PERIMETER SECURITY BEST PRACTICES

- ✓ ENABLE AWS SHIELD FOR ALL EXPOSED APPLICATIONS.
- ✓ USE AWS WAF WITH MANAGED AND CUSTOM RULES.
- ✓ CENTRALIZE FIREWALL RULE MANAGEMENT WITH FIREWALL MANAGER.
- ✓ USE ROUTE 53 RESOLVER DNS FIREWALL TO BLOCK MALICIOUS DOMAINS.
- ✓ KEEP TRAFFIC PRIVATE USING VPN, DIRECT CONNECT, AND PRIVATELINK.
- ✓ INSPECT TRAFFIC WITH AWS NETWORK FIREWALL BEFORE IT REACHES WORKLOADS.
- ✓ LOG EVERYTHING AND MONITOR WITH AMAZON CLOUDWATCH AND SECURITY HUB.

COMMON MISTAKES TO AVOID

- ✗ EXPOSING APPLICATIONS WITHOUT WAF OR SHIELD.
- ✗ ALLOWING ALL TRAFFIC IN SECURITY GROUPS.
- ✗ NOT FILTERING DNS TRAFFIC.
- ✗ SENDING TRAFFIC TO THE INTERNET UNNECESSARILY.
- ✗ NO CENTRAL MANAGEMENT OF FIREWALL POLICIES.
- ✗ SKIPPING TRAFFIC INSPECTION FOR ENCRYPTED TRAFFIC.
- ✗ NOT MONITORING AND ALERTING ON THREATS.



A STRONG PERIMETER IS
YOUR FIRST LINE OF DEFENSE.
LAYER IT. MANAGE IT. MONITOR IT.
PROTECT WHAT MATTERS.



PREVENT
ATTACKS



DETECT
THREATS



INSPECT
TRAFFIC



SECURE
CONNECTIVITY



PROTECT
WORKLOADS

ENCRYPTION AND AWS KMS

PROTECT YOUR DATA. CONTROL YOUR KEYS. SECURE YOUR AWS ENVIRONMENT.

1 ENCRYPTION AT REST



- Protects data stored on disks, databases, backups, and object storage.
- Use AWS KMS for managing encryption keys securely.
- Examples: S3, EBS, RDS, DynamoDB, EFS, Redshift.

2 ENCRYPTION IN TRANSIT



- Protects data while it moves between clients, services, and across networks.
- Use TLS/SSL for all connections.
- Examples: HTTPS, TLS for API calls, VPN, Direct Connect.

3 AWS KMS (KEY MANAGEMENT SERVICE)



- Fully managed service to create, manage, and control encryption keys.
- Integrated with AWS services.
- Centralized key management.
- FIPS 140-2 Level 2 validated.

HOW AWS KMS WORKS



YOU CONTROL



4 KMS KEY CONCEPTS

KMS KEYS



- Symmetric keys (AES-256) for most use cases.
- Customer managed keys (CMKs) give you full control.

KEY POLICIES



- Control who can use the key and how.
- Define permissions for IAM users, roles, and accounts.

GRANTS



- Allow temporary, permission to use a key.
- More secure than sharing IAM policies.

ENVELOPE ENCRYPTION



- KMS encrypts a data key (small key).
- Data key is used to encrypt the actual large data.

KEY ROTATION



- Automatically rotate keys annually.
- Keeps data secure with minimal effort.

CROSS-ACCOUNT ENCRYPTION



- Share keys securely across AWS accounts.
- Use key policies and grants.

5 ENVELOPE ENCRYPTION FLOW



ONLY THE KMS KEY CAN DECRYPT THE DATA KEY, WHICH CAN THEN DECRYPT THE DATA.



BEST PRACTICES

- ✓ ENABLE ENCRYPTION BY DEFAULT ON ALL SUPPORTED SERVICES.
- ✓ USE CUSTOMER MANAGED KEYS (CMKS) FOR SENSITIVE DATA.
- ✓ APPLY LEAST PRIVILEGE IN KEY POLICIES AND GRANTS.
- ✓ ENABLE AUTOMATIC KEY ROTATION.
- ✓ MONITOR KMS KEY USAGE WITH CLOUDTRAIL AND CLOUDWATCH.
- ✓ USE ALIASES FOR EASY KEY MANAGEMENT.



COMMON MISTAKES TO AVOID

- ✗ USING THE AWS ACCOUNT ROOT USER TO MANAGE KEYS.
- ✗ SHARING KEYS WIDELY USING "*" IN KEY POLICIES.
- ✗ NOT ROTATING KEYS REGULARLY.
- ✗ STORING UNENCRYPTED DATA.
- ✗ NOT MONITORING KEY USAGE AND ACCESS.



**ENCRYPT EVERYTHING.
TRUST FEW.**



**STRONG ENCRYPTION STARTS WITH STRONG KEY MANAGEMENT.
AWS KMS GIVES YOU CONTROL, VISIBILITY, AND SECURITY AT SCALE.**



SECRETS AND CREDENTIAL MANAGEMENT

STORE SECURELY. ACCESS SAFELY. ROTATE REGULARLY. NEVER HARDCODE.

1 AWS SECRETS MANAGER



- Fully managed service for storing and managing secrets.
- Automatically encrypts secrets using AWS KMS.
- Integrates with many AWS services and third-party apps.
- Built-in rotation support.
- Audit and monitor access with AWS CloudTrail.

2 SYSTEMS MANAGER PARAMETER STORE



- Store configuration data and secrets as parameters.
- Standard parameters – free.
- SecureString parameters – encrypted with KMS.
- Hierarchical structure for easy organization.
- Use in applications, automation, and scripts.
- Great for small secrets and configuration values.

3 SECRETS ROTATION



- Rotate secrets regularly to reduce risk.
- Automated rotation with AWS Lambda.
- Supported for databases, APIs, and more.
- Custom rotation for your own applications.

ROTATION FLOW



SCHEDULE
ROTATION



CREATE NEW
SECRET



UPDATE
APPLICATIONS



REMOVE OLD
SECRET

4 APPLICATION CREDENTIALS



- Store app credentials in Secrets Manager or Parameter Store.
- Use IAM roles for applications whenever possible.
- Never hardcode keys, tokens, or passwords.

5 DATABASE CREDENTIALS



- Store DB usernames and passwords in Secrets Manager.
- Enable automatic rotation for supported databases.
- Use IAM database authentication where possible.

6 API SECRETS



- Store API keys, tokens, and client secrets securely.
- Rotate keys regularly.
- Restrict access using IAM policies.
- Monitor usage and set alerts.

7 AVOIDING HARDCODED CREDENTIALS



- Never commit secrets to code repositories.
- Use environment variables or secure services.
- Scan code for exposed secrets.
- Educate your team and enforce policies.

8 TEMPORARY CREDENTIALS (BEST PRACTICE)



- Use IAM roles to get temporary credentials.
- Temporary credentials reduce the risk of long-term access.
- AWS STS provides short-lived tokens.
- Ideal for EC2, Lambda, ECS, EKS, and applications.
- Never store long-term access keys in code or config.

HOW TEMPORARY CREDENTIALS WORK



USER / APP



AWS STS
Assume Role
(Short-lived)



TEMPORARY
CREDENTIALS
Use Temporary
Credentials

BEST PRACTICES



- ✓ Store all secrets in Secrets Manager or Parameter Store.
- ✓ Enable automatic rotation for supported secrets.
- ✓ Use IAM roles and temporary credentials.
- ✓ Apply least privilege access to secrets.
- ✓ Monitor and audit secret access regularly.

COMMON MISTAKES TO AVOID



- ✗ Hardcoding secrets in code or configuration files.
- ✗ Using long-term access keys.
- ✗ Overly broad access permissions.
- ✗ Not rotating secrets regularly.
- ✗ Storing secrets in plain text.



SECURE YOUR SECRETS. PROTECT YOUR APPLICATIONS.
GOOD CREDENTIAL MANAGEMENT IS GREAT SECURITY.



SECURING AMAZON S3

PROTECT YOUR DATA. CONTROL ACCESS. PREVENT EXPOSURE.

1 BLOCK PUBLIC ACCESS



- Enable S3 Block Public Access at the account level.
- Block access to buckets and objects granted through public policies.
- Block public and cross-account access through any ACL.
- Review and test before enabling in production.

2 BUCKET POLICIES



- Use bucket policies to control access.
- Prefer bucket policies over ACLs.
- Use least privilege principle.
- Deny by default, allow only what is required.

3 IAM ACCESS



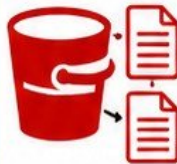
- Use IAM users, groups, or roles for access.
- Apply least privilege permissions.
- Use condition keys (e.g., `aws:SourceVpce`, `aws:SecureTransport`).
- Avoid using the root user.

4 ENCRYPTION



- Enable encryption at rest using SSE-S3, SSE-KMS, or SSE-C.
- Use AWS KMS CMKs for better control and auditing.
- Enforce encryption in bucket policies.

5 VERSIONING



- Enable versioning to protect against accidental deletions and overwrites.
- Easily restore previous versions of objects.
- Works with MFA Delete for extra protection.

6 OBJECT LOCK



- Use Object Lock to prevent objects from being deleted or overwritten.
- Choose Governance or Compliance mode.
- Ideal for compliance and legal requirements.

7 ACCESS LOGGING



- Enable server access logging to track requests.
- Store logs in a separate, secure logging bucket.
- Helps with monitoring, auditing, and forensics.

8 S3 ACCESS POINTS



- Use Access Points to manage data access at scale.
- Simplify permissions for shared datasets.
- Integrate with VPCs for private access.

9 PREVENTING ACCIDENTAL PUBLIC EXPOSURE



- Regularly scan for public buckets using tools.
- Use AWS Config rules and Security Hub.
- Continuously monitor and review bucket permissions.
- Automate alerts for public exposure.

SECURE S3 BUCKET CHECKLIST



BLOCK PUBLIC ACCESS ENABLED



BUCKET POLICY CONFIGURED



IAM LEAST PRIVILEGE ACCESS APPLIED



ENCRYPTION ENABLED



VERSIONING ENABLED



OBJECT LOCK CONFIGURED



ACCESS LOGGING ENABLED



ACCESS POINTS IN USE



BEST PRACTICES



- ✓ ENABLE S3 BLOCK PUBLIC ACCESS AT THE ACCOUNT LEVEL.
- ✓ USE BUCKET POLICIES AND LEAST PRIVILEGE IAM PERMISSIONS.
- ✓ ENCRYPT ALL DATA AT REST AND IN TRANSIT.
- ✓ ENABLE VERSIONING AND MFA DELETE.
- ✓ MONITOR ACCESS LOGS AND CONFIG RULES REGULARLY.
- ✓ AUTOMATE ALERTS FOR PUBLIC ACCESS AND POLICY CHANGES.



COMMON MISTAKES

- ✗ LEAVING S3 BUCKETS PUBLIC.
- ✗ USING WIDE OPEN POLICIES (* PRINCIPAL).
- ✗ DISABLING BLOCK PUBLIC ACCESS.
- ✗ NOT ENCRYPTING SENSITIVE DATA.
- ✗ NOT ENABLING VERSIONING.
- ✗ NOT MONITORING FOR PUBLIC EXPOSURE.



SECURE YOUR S3 BUCKETS TODAY.
PROTECT DATA. PREVENT BREACHES. STAY COMPLIANT.



SECURING EC2 AND COMPUTE WORKLOADS

BUILD SECURE. PROTECT WORKLOADS. OPERATE WITH CONFIDENCE.

1 IAM INSTANCE ROLES



- Use IAM roles instead of long-term access keys.
- Attach least privilege policies to roles.
- Applications get temporary credentials automatically.
- Never store credentials in code or files.

2 SECURITY GROUPS



- Act as a stateful firewall at the instance level.
- Allow only required inbound traffic.
- Restrict outbound traffic to what is necessary.
- Use separate SGs for different tiers.

3 IMDSv2 (INSTANCE METADATA SERVICE v2)



- Enforce IMDSv2 on all instances.
- Prevent SSRF attacks and credential theft.
- Set hop limit to 1.
- Disable IMDSv1 at launch and via organization policy.

4 EBS ENCRYPTION



- Encrypt EBS volumes using AWS KMS.
- Use customer managed CMKs for better control.
- Encrypt snapshots as well.
- Enforce encryption by default.

5 SYSTEMS MANAGER



- Use SSM for secure instance management.
- No need for SSH or bastion hosts.
- Run commands, collect logs, and automate tasks securely.

6 PATCH MANAGEMENT



- Use AWS Systems Manager Patch Manager.
- Keep OS and applications up to date.
- Automate patch compliance.
- Schedule patch windows and reboots.

7 HARDENED AMIS



- Use minimal, hardened AMIs as a baseline.
- Remove unnecessary software and accounts.
- Keep AMIs updated and scanned.
- Build reusable golden AMIs.

8 REMOVING UNNECESSARY PUBLIC ACCESS



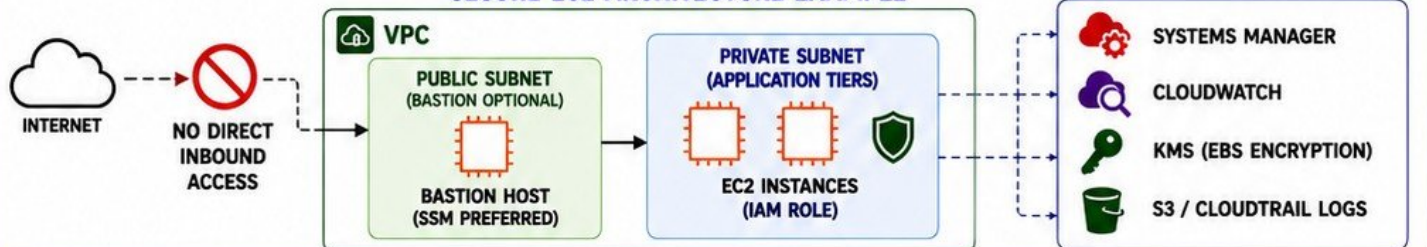
- Don't assign public IPs unless necessary.
- Use private subnets for workloads.
- Access via SSM, VPN, or bastion when needed.
- Restrict inbound access to only required sources.

9 INSTANCE SECURITY MONITORING



- Monitor with Amazon CloudWatch metrics and alarms.
- Enable detailed monitoring.
- Use GuardDuty for threat detection.
- Send logs to CloudWatch and/or SIEM.

SECURE EC2 ARCHITECTURE EXAMPLE



BEST PRACTICES

- ✓ USE IAM ROLES – NEVER ACCESS KEYS.
- ✓ ENFORCE IMDSv2 ON ALL INSTANCES.
- ✓ KEEP SECURITY GROUPS TIGHT.
- ✓ ENCRYPT ALL EBS VOLUMES AND SNAPSHOTS.
- ✓ USE SSM FOR MANAGEMENT.
- ✓ PATCH REGULARLY.
- ✓ USE PRIVATE SUBNETS.
- ✓ MONITOR AND ALERT ACTIVELY.

COMMON MISTAKES TO AVOID

- ✗ USING ACCESS KEYS ON EC2 INSTANCES.
- ✗ ALLOWING 0.0.0.0/0 IN SECURITY GROUPS.
- ✗ DISABLING OR IGNORING IMDSv2.
- ✗ RUNNING UNPATCHED INSTANCES.
- ✗ EXPOSING SSH/RDP TO THE INTERNET.
- ✗ USING DEFAULT AMIs.
- ✗ NOT ENCRYPTING EBS VOLUMES.
- ✗ NOT MONITORING INSTANCES.



SECURE EVERY INSTANCE. HARDEN EVERY LAYER.
A SECURE WORKLOAD IS A RESILIENT WORKLOAD.



DATABASE SECURITY

SECURE YOUR DATA. PROTECT YOUR DATABASES.

1 RDS SECURITY



- Use RDS managed services for built-in security.
- Keep engines and minor versions up to date.
- Disable public access.
- Use least privilege access.

2 PRIVATE DATABASE SUBNETS



- Deploy RDS in private subnets across multiple AZs.
- No direct access from the internet.
- Access only through application tier.

3 SECURITY GROUPS



- Use SGs as stateful firewalls.
- Allow access only from application security groups.
- Restrict ports to only what is required.

4 ENCRYPTION



- Encrypt RDS storage using AWS KMS.
- Encrypt backups, snapshots, and read replicas.
- Enforce encryption.

5 IAM DATABASE AUTHENTICATION



- Use IAM for authentication where supported.
- Eliminates password management.
- Map IAM users to database users.

6 SECRETS MANAGER INTEGRATION



- Store database credentials in AWS Secrets Manager.
- Enable automatic rotation for supported engines.
- Update applications to use secrets securely.

7 BACKUP SECURITY



- Enable automated backups.
- Encrypt backups using KMS.
- Store backups in private subnets.
- Regularly test restore processes.

8 SNAPSHOT SECURITY



- Encrypt all snapshots.
- Restrict snapshot sharing.
- Copy important snapshots to another region for protection.

9 DATABASE AUDITING

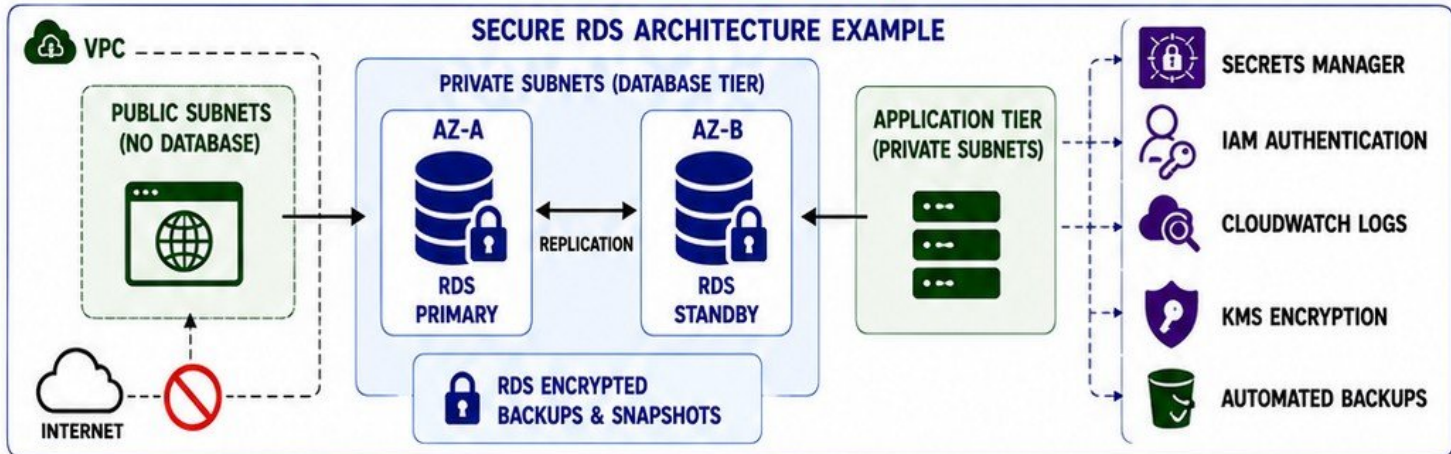


- Enable RDS audit logging.
- Send logs to CloudWatch Logs or S3.
- Monitor and alert on suspicious activities.

10 OTHER BEST PRACTICES



- Use latest engine versions.
- Apply patches promptly.
- Use parameter groups to enforce security settings.
- Monitor performance and security.



SECURITY CHECKLIST

- ✓ RDS deployed in private subnets.
- ✓ No public access to the database.
- ✓ Security groups restrict access.
- ✓ Encryption enabled (KMS).
- ✓ Backups encrypted and tested.
- ✓ Audit logging enabled.
- ✓ Secrets stored in Secrets Manager.
- ✓ IAM authentication used (if supported).

COMMON MISTAKES TO AVOID

- ✗ Exposing RDS to the internet.
- ✗ Using overly permissive security groups.
- ✗ Not encrypting databases and backups.
- ✗ Storing credentials in code or plain text.
- ✗ Not enabling audit logs and monitoring.



SECURE YOUR DATABASES TODAY.
PROTECT DATA. ENSURE AVAILABILITY. MAINTAIN TRUST.



CLOUDTRAIL AND SECURITY LOGGING

LOG EVERYTHING. PROTECT LOGS. INVESTIGATE EVERYTHING.

1 AWS CLOUDTRAIL



- Records AWS account activity and API calls.
- Enables governance, compliance, and operational visibility.
- Provides event history for troubleshooting and security analysis.

2 MANAGEMENT EVENTS



- Log management operations.
- Examples: Create User, RunInstances, AttachPolicy.
- Captured by default.
- Essential for security monitoring and compliance.

3 DATA EVENTS



- Log resource-level operations.
- Examples: S3 object access, Lambda invocation.
- Must be enabled explicitly.
- Useful for detecting data access.

4 ORGANIZATION TRAILS



- Create trails for the entire AWS Organization.
- Capture events from all accounts.
- Simplify auditing and compliance across accounts.

5 CLOUDWATCH LOGS



- CloudTrail delivers logs to CloudWatch Logs.
- Enables real-time monitoring and alerting.
- Retain, search, and analyze logs easily.

6 CENTRALIZED LOGGING



- Store logs in a central logging account.
- Use CloudTrail Organization trails.
- Aggregate logs from all regions and accounts.

7 LOG INTEGRITY



- Enable log file validation.
- Logs are signed using SHA-256.
- Prevent tampering and unauthorized changes.
- Ensure audit log trustworthiness.

8 PROTECTING AUDIT LOGS



- Store logs in a dedicated S3 bucket.
- Enable S3 Object Lock (WORM).
- Block public access.
- Restrict bucket access with least privilege.

9 INVESTIGATING API ACTIVITY



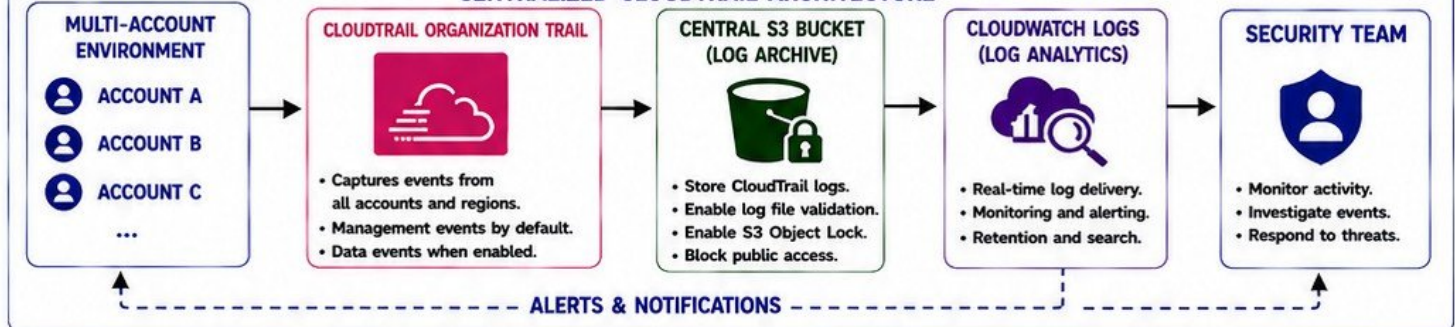
- Search and analyze events in CloudTrail.
- Identify who, what, when, where, and from where.
- Detect suspicious or unauthorized activity.

10 BEST PRACTICES OVERVIEW



- Enable CloudTrail in all regions.
- Use Organization trails.
- Log management and data events.
- Protect logs and monitor continuously.

CENTRALIZED CLOUDTRAIL ARCHITECTURE



WHAT CLOUDTRAIL LOGS



WHO
User/Role



WHAT
API Action



WHEN
Timestamp



WHERE
Source IP



RESOURCE
Affected Resource

BEST PRACTICES

- ✓ ENABLE CLOUDTRAIL IN ALL REGIONS.
- ✓ USE ORGANIZATION TRAILS FOR COMPLETE VISIBILITY. LOG MANAGEMENT AND DATA EVENTS.
- ✓ STORE LOGS IN A CENTRAL, SECURE S3 BUCKET.
- ✓ ENABLE LOG FILE VALIDATION AND S3 OBJECT LOCK.
- ✓ MONITOR LOGS IN CLOUDWATCH AND SET ALERTS.

COMMON USE CASES



DETECT
UNAUTHORIZED
ACCESS



INVESTIGATE
SECURITY
INCIDENTS



ENSURE
COMPLIANCE
& AUDITS



MONITOR
ADMIN ACTIVITY
& CHANGES

COMMON MISTAKES TO AVOID

- ✗ DISABLING CLOUDTRAIL OR NOT ENABLING ALL REGIONS.
- ✗ NOT LOGGING DATA EVENTS (S3, LAMBDA, ETC.).
- ✗ STORING LOGS IN THE SAME ACCOUNT AS THE WORKLOADS.
- ✗ NOT PROTECTING LOGS FROM DELETION OR MODIFICATION.
- ✗ NOT MONITORING CLOUDTRAIL LOGS REGULARLY.



**YOU CAN'T PROTECT WHAT YOU DON'T LOG.
CLOUDTRAIL GIVES YOU VISIBILITY. LOGS GIVE YOU EVIDENCE.**



AWS CONFIG AND CONTINUOUS COMPLIANCE

MONITOR CONTINUOUSLY. DETECT DRIFT. STAY COMPLIANT.

1 AWS CONFIG



- AWS Config records and evaluates configuration changes.
- Provides a detailed view of your AWS resources.
- Helps in security, governance, and compliance.

2 CONFIGURATION HISTORY



- Records resource configurations over time.
- View full change history and relationships.
- Retain configuration snapshots for auditing.

3 CONFIG RULES



- Evaluate resources against desired configurations.
- AWS managed rules or custom rules.
- Automatically evaluate and flag noncompliant resources.

4 CONFORMANCE PACKS



- Predefined sets of rules for best practices and compliance.
- Supports AWS, industry, and custom packs.
- Deploy consistently across accounts and regions.

5 DETECTING CONFIGURATION DRIFT



- Identify deviations from desired configurations.
- Detect unauthorized changes quickly.
- Maintain a secure and consistent environment.

6 COMPLIANCE MONITORING



- Continuously monitor resource compliance.
- Track compliance status over time.
- Generate compliance reports and dashboards.

7 AUTOMATED REMEDIATION



- Use AWS Systems Manager Automation to fix issues.
- Trigger remediation based on Config rule violations.
- Reduce manual effort and MTTR.

8 MULTI-ACCOUNT AGGREGATION



- Aggregate Config data across multiple accounts.
- Centralize compliance visibility.
- Simplify governance at scale.

9 CENTRALIZED DASHBOARDS



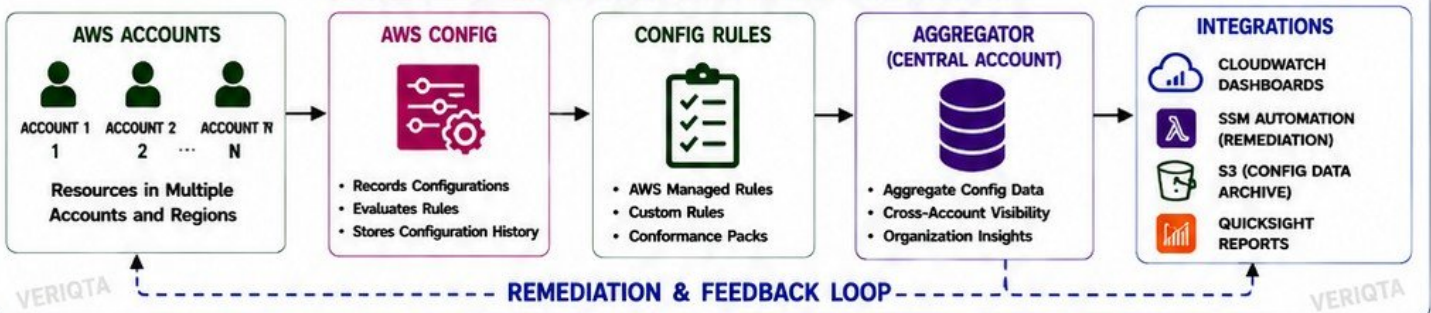
- View compliance and resource data in one place.
- Custom dashboards using Amazon QuickSight.
- Share insights with stakeholders.

10 BEST PRACTICES



- Enable AWS Config in all regions.
- Use organization aggregation.
- Apply conformance packs.
- Automate remediation.
- Review compliance regularly.

AWS CONFIG CONTINUOUS COMPLIANCE ARCHITECTURE



BENEFITS OF AWS CONFIG



IMPROVE SECURITY AND GOVERNANCE



ACHIEVE AND MAINTAIN COMPLIANCE



REDUCE RISK WITH CONTINUOUS MONITORING



AUTOMATE REMEDIATION AND REDUCE MTTR



SCALE COMPLIANCE ACROSS MULTIPLE ACCOUNTS

BEST PRACTICES



- ✓ ENABLE AWS CONFIG IN ALL REGIONS.
- ✓ USE ORGANIZATION AGGREGATORS.
- ✓ DEPLOY CONFORMANCE PACKS.
- ✓ AUTOMATE REMEDIATION FOR FAILING RULES.
- ✓ MONITOR COMPLIANCE DASHBOARDS REGULARLY.
- ✓ ARCHIVE CONFIG DATA IN S3 SECURELY.



COMMON MISTAKES TO AVOID

- ✗ NOT ENABLING AWS CONFIG IN ALL ACCOUNTS/REGIONS.
- ✗ IGNORING CONFIG RULE VIOLATIONS.
- ✗ NOT USING AGGREGATORS FOR CENTRAL VISIBILITY.
- ✗ NOT AUTOMATING REMEDIATION.
- ✗ NOT REVIEWING COMPLIANCE REPORTS.



CONTINUOUS COMPLIANCE IS A PROCESS, NOT A CHECKLIST.
USE AWS CONFIG TO STAY COMPLIANT EVERY SINGLE DAY.



THREAT DETECTION WITH AMAZON GUARDDUTY

CONTINUOUS THREAT DETECTION. SMARTER SECURITY.

DATA SOURCES

- AWS CLOUDTRAIL**
API activity logs
- VPC FLOW LOGS**
Network traffic
- S3 DATA EVENTS**
Object-level activity
- DNS LOGS**
DNS queries
- EKS AUDIT LOGS**
Kubernetes activity

1. GUARDDUTY ARCHITECTURE

AMAZON GUARDDUTY



- Continuously analyzes data
- Detects threats and anomalies
- Uses ML, threat intelligence and behavioral analysis
- Generates security findings

OUTPUTS & INTEGRATIONS

- SECURITY FINDINGS**
Detailed threat findings
- AMAZON EVENTBRIDGE**
Automate actions
- AWS LAMBDA**
Automated remediation
- S3 / CLOUDWATCH LOGS**
Store findings & logs
- AMAZON SECURITY HUB**
Centralized visibility

2. GUARDDUTY THREAT FINDINGS

SUSPICIOUS API ACTIVITY



Detects unusual or unexpected API calls or usage patterns.

COMPROMISED CREDENTIALS



Identifies the use of credentials exposed or used in malicious activity.

EC2 THREATS



Detects malware, crypto mining, unauthorized access, and other EC2 risks.

S3 PROTECTION



Identifies unusual access to S3 buckets and sensitive data.

RUNTIME MONITORING



Monitors runtime behavior of EC2, EKS, and containers for threats.

3. FINDING SEVERITY

	CRITICAL	High confidence of active threat or compromise.
	HIGH	Likely threat or suspicious activity with high impact.
	MEDIUM	Potential threat that may require investigation.
	LOW	Unusual activity with low risk or informational.
	INFORMATIONAL	Informational finding for visibility and awareness.

4. INVESTIGATION WORKFLOW

1		DETECT	GuardDuty detects abnormal or malicious activity and generates a security finding.
2		REVIEW	Review finding details: resource, type, time, account, and severity.
3		INVESTIGATE	Analyze related resources, logs, IAM activity, and network flows.
4		RESPOND	Take action: isolate resources, revoke access, or block activity.
5		REMEDiate & MONITOR	Remediate the issue and monitor to ensure the threat is resolved.

BEST PRACTICES

- ✓ Enable GuardDuty in all regions and accounts.
- ✓ Review findings daily.
- ✓ Integrate with Security Hub for centralized visibility.
- ✓ Automate response using EventBridge and Lambda.
- ✓ Maintain least privilege for all access.
- ✓ Use Runtime Monitoring for workload protection.

WHY GUARDDUTY?

- ✓ Intelligent threat detection
- ✓ Continuous monitoring
- ✓ Low setup, high impact
- ✓ Built-in threat intelligence
- ✓ Strengthens your security posture

SECURITY HUB AND CENTRALIZED SECURITY OPERATIONS

CENTRALIZE. CORRELATE. PRIORITIZE. RESPOND.

1 AWS SECURITY HUB CSPM



**AWS
SECURITY HUB**

- Cloud Security Posture Management (CSPM)
- Continuously checks your AWS environment against security best practices and standards
- Aggregates findings from multiple AWS services and partner solutions
- Provides a unified security view across accounts and regions

2 SECURITY STANDARDS

- ☒ AWS Foundational Security Best Practices
- ☒ CIS AWS Foundations Benchmark
- ☒ NIST SP 800-53
- ☒ PCI DSS
- ☒ ISO 27001

MORE STANDARDS SUPPORTED

3 FINDINGS



- Aggregates and normalizes findings from AWS services and partners
- Provides context: resource, account, region, severity, and time
- Tracks compliance status continuously
- Supports automated remediation

4 AWS FOUNDATIONAL SECURITY BEST PRACTICES



- Predefined checks for the most critical security controls
- Covers identity, access, logging, encryption, networking, and more
- Helps you maintain a strong security baseline

5 SERVICE INTEGRATIONS

GUARDDUTY INTEGRATION



Ingests GuardDuty threat findings for central visibility and correlation.

INSPECTOR INTEGRATION



Ingests vulnerability findings from Amazon Inspector for EC2, ECR, Lambda, and more.

CONFIG INTEGRATION



Ingests AWS Config compliance and configuration findings continuously.

OTHER PARTNER INTEGRATIONS



Supports third-party tools via AWS Security Hub APIs and custom integrations.

6 FINDING PRIORITIZATION

	CRITICAL	High risk with active threat or critical impact
	HIGH	Likely to be exploited or high impact
	MEDIUM	Moderate risk or potential misconfiguration
	LOW	Low risk with minimal impact
	INFORMATIONAL	Informational or best practice observations

HIGHEST PRIORITY



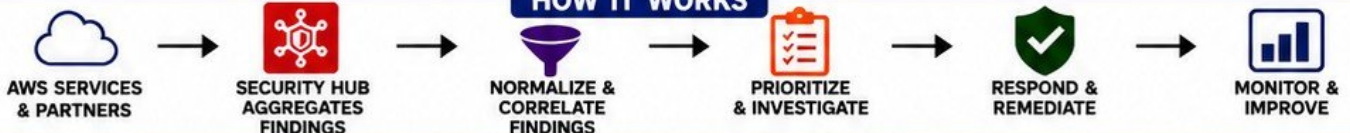
LOWEST PRIORITY

7 CENTRAL SECURITY OPERATIONS



- Centralized dashboard for all security findings
- Unified view across accounts, regions, and services
- Automated alerts and notifications
- Integration with SIEM, SOAR, and ITSM tools
- Track trends, measure compliance, and improve security posture
- Faster detection, response, and continuous improvement

HOW IT WORKS



VERIQTA - ENGINEERING EXCELLENCE

VULNERABILITY AND WORKLOAD SECURITY

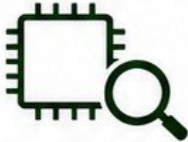
DISCOVER. ASSESS. PRIORITIZE. REMEDIATE.



AMAZON INSPECTOR

Continuous vulnerability management for your workloads.

1 EC2 VULNERABILITY SCANNING



- Scans EC2 instances for OS vulnerabilities.
- Detects missing security updates.
- Identifies network and software risks.

2 CONTAINER IMAGE SCANNING



- Scans container images in Amazon ECR.
- Detects vulnerabilities in OS packages and application libraries.
- Supports CI/CD integration.

3 ECR SECURITY



- Continuous scanning of images in ECR.
- Blocks images with critical and high vulnerabilities.
- Enforces secure image lifecycle.

4 LAMBDA VULNERABILITY ASSESSMENT



- Scans Lambda function code and libraries.
- Detects vulnerable dependencies.
- Identifies insecure configurations.

5 PATCH MANAGEMENT



- Identifies missing operating system patches.
- Works with SSM Patch Manager.
- Reduces exposure to known threats.

6 DEPENDENCY VULNERABILITIES



- Detects vulnerable libraries and packages.
- Supports multiple package ecosystems.
- Monitors for new vulnerability disclosures.

7 REMEDIATION PRIORITIZATION



- Risk score based on CVSS severity.
- Context-aware prioritization.
- Focus on what matters most first.

8 CONTINUOUS PROTECTION



- Continuous scanning and reassessment.
- Real-time visibility into new risks.
- Integrates with Security Hub.

HOW AMAZON INSPECTOR WORKS



AWS WORKLOADS
EC2, ECR, Lambda
and other resources



AMAZON INSPECTOR
Automated discovery
and continuous scanning



FINDINGS
Detailed vulnerability
findings and insights



PRIORITIZE
Risk-based
prioritization



REMEDiate
Take action and
verify remediation

SUPPORTED WORKLOADS



AMAZON
EC2



ECR
IMAGES



AWS
LAMBDA



ON-PREMISES
& HYBRID

BENEFITS

- ✓ IMPROVE SECURITY POSTURE
- ✓ REDUCE RISK AND EXPOSURE
- ✓ AUTOMATE VULNERABILITY MANAGEMENT
- ✓ SPEED UP REMEDIATION
- ✓ INTEGRATE WITH AWS SECURITY SERVICES

DEVSECOPS AND CI/CD SECURITY

BUILD SECURELY. TEST CONTINUOUSLY. DEPLOY CONFIDENTLY.



SECURITY IN EVERY STAGE OF YOUR CI/CD PIPELINE

1 SECURITY IN CI/CD



- ✓ Integrate security into every stage.
- ✓ Shift security left.
- ✓ Automate security checks.
- ✓ Fail fast, fix early.

2 IAM ROLES FOR PIPELINES



- ✓ Use least-privilege IAM roles.
- ✓ No long-term credentials.
- ✓ One role per pipeline.
- ✓ Limit permissions to what's needed.

3 SECRETS PROTECTION



- ✓ Store secrets in AWS Secrets Manager or SSM.
- ✓ Never hardcode credentials.
- ✓ Rotate secrets regularly.

4 INFRASTRUCTURE-AS-CODE SCANNING



- ✓ Scan IaC templates (CloudFormation, Terraform, CDK).
- ✓ Detect misconfigurations and risky resources.
- ✓ Use tools like cfn-nag, Checkov, or tfsec.

5 DEPENDENCY SCANNING



- ✓ Scan code dependencies for known vulnerabilities.
- ✓ Use tools like Snyk, Trivy, or Dependency-Check.
- ✓ Keep dependencies up to date.

6 CONTAINER SCANNING



- ✓ Scan container images for vulnerabilities.
- ✓ Use Amazon ECR scanning, Trivy, or Clair.
- ✓ Block images with critical vulnerabilities.

7 ARTIFACT PROTECTION



- ✓ Store artifacts in secure repositories.
- ✓ Enable encryption at rest.
- ✓ Use integrity checksums and signing.
- ✓ Limit access to artifacts.

8 DEPLOYMENT PERMISSIONS



- ✓ Restrict who can deploy.
- ✓ Use approval workflows.
- ✓ Separate build and deploy roles.
- ✓ Enforce least privilege in production.

9 SECURITY GATES



- ✓ Automated checks must pass.
- ✓ Block high-risk code from progressing.
- ✓ Include SAST, DAST, IaC, and dependency scans.
- ✓ No bypass in production.

10 SUPPLY-CHAIN SECURITY



- ✓ Verify source integrity.
- ✓ Pin dependencies and base images.
- ✓ Sign artifacts and container images.
- ✓ Use trusted registries and repositories.

CI/CD SECURITY PIPELINE



CODE
Security linting & scanning



BUILD
Artifact protection



SCAN
SAST, DAST, IaC & dependency



SECURITY GATE
Block risky builds



PACKAGE
Secure artifacts



DEPLOY
Least privilege access



MONITOR
Detect and respond

TOOLS YOU CAN USE



GitHub Actions



GitLab CI/CD



Jenkins



AWS CodePipeline

BEST PRACTICES

- ✓ Shift security left in the pipeline.
- ✓ Automate security scanning.
- ✓ Use least privilege for all pipeline roles.
- ✓ Protect secrets and artifacts.
- ✓ Verify dependencies.
- ✓ Monitor and respond to threats.

SECURITY OUTCOMES

- ✓ Fewer vulnerabilities.
- ✓ Safer deployments.
- ✓ Stronger supply chain.
- ✓ Production-ready security.



**SECURE TODAY.
PROTECT TOMORROW.
SHIP WITH CONFIDENCE.**

VERIQTA – ENGINEERING EXCELLENCE



veriqta



veriqta



veriqta



veriqta



veriqta

AWS INCIDENT RESPONSE AND FORENSICS

DETECT. TRIAGE. CONTAIN. INVESTIGATE. RECOVER. IMPROVE.



PREPARE TODAY. RESPOND CONFIDENTLY.

A STRUCTURED APPROACH TO SECURE, CONTAIN, AND RECOVER FROM SECURITY INCIDENTS IN AWS.



1 DETECTION

- Monitor and detect suspicious activity.
- Use GuardDuty, Security Hub, CloudTrail, CloudWatch.
- Validate alerts and confirm impact.



6 ISOLATING COMPROMISED WORKLOADS

- Remove instances from load balancers.
- Quarantine EC2, containers, or Lambda functions.
- Restrict security groups and network access.
- Disable or rebuild if necessary.



2 TRIAGE

- Assess the alert and scope the impact.
- Determine severity and priority.
- Identify affected accounts, resources, and users.



7 CLOUDTRAIL INVESTIGATION

- Review CloudTrail for API activity.
- Identify unusual or unauthorized actions.
- Trace the source IP, user, and time.
- Export and store logs securely.



3 CONTAINMENT

- Stop further damage immediately.
- Disable compromised credentials.
- Isolate affected resources.
- Block malicious IPs and revoke access.



8 EVIDENCE PRESERVATION

- Preserve logs, snapshots, and artifacts.
- Do not delete or modify evidence.
- Store evidence securely in S3 (encrypted and access controlled).
- Document everything.



4 INVESTIGATION

- Investigate the root cause.
- Analyze logs, events, and configuration changes.
- Use CloudTrail, Config, VPC Flow Logs, and application logs.
- Create a clear timeline.



9 RECOVERY

- Restore systems and services.
- Patch vulnerabilities.
- Validate integrity of workloads.
- Monitor closely after recovery.



5 CREDENTIAL COMPROMISE

- Identify exposed or abused credentials.
- Rotate passwords, keys, and tokens.
- Enforce MFA and strong policies.
- Review IAM access and permissions.



10 POST-INCIDENT REVIEW

- Conduct a lessons learned meeting.
- Document root cause and impact.
- Improve controls and automate detections.
- Update runbooks and security policies.

KEY AWS SERVICES FOR INCIDENT RESPONSE



AMAZON GUARD DUTY
Threat detection and monitoring



AWS SECURITY HUB
Centralized security findings



AMAZON S3
Store logs and evidence securely



AMAZON CLOUDTRAIL
API activity and auditing



AMAZON CLOUDWATCH
Monitoring and alerting



AWS CONFIG
Resource changes and compliance



AWS IAM
Access control and roles

INCIDENT RESPONSE BEST PRACTICES

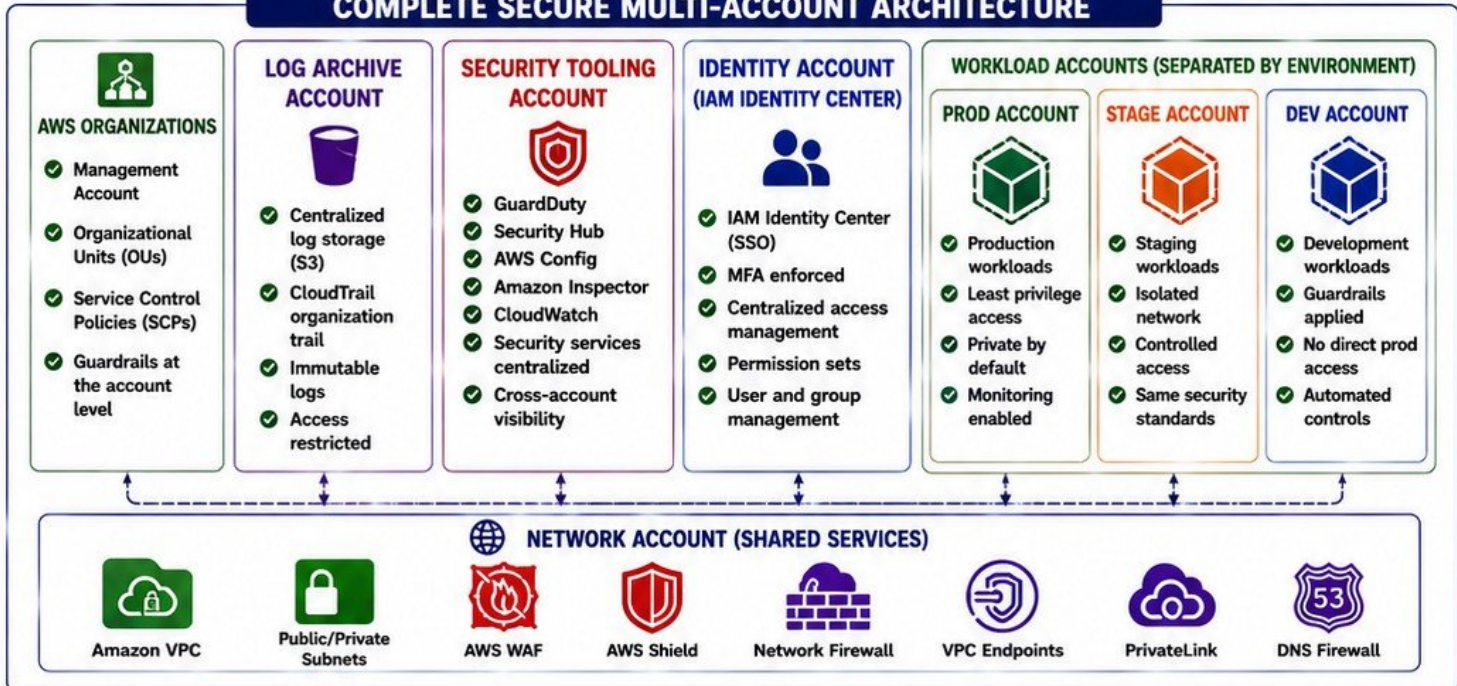
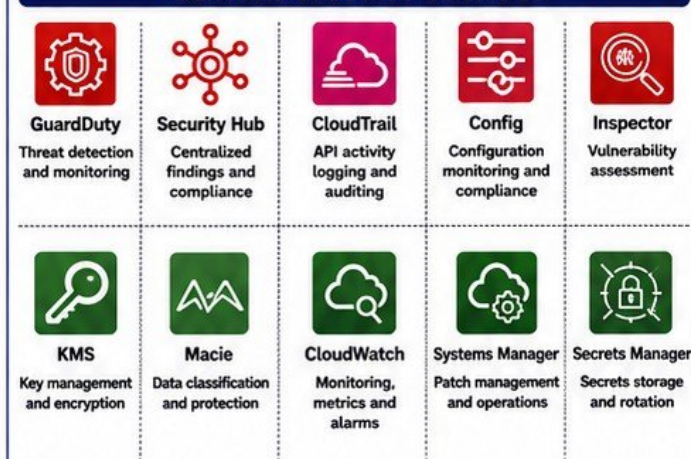
- ✓ AUTOMATE DETECTIONS AND ALERTS
- ✓ FOLLOW LEAST PRIVILEGE ACCESS
- ✓ ENABLE MFA FOR ALL USERS
- ✓ CENTRALIZE LOGGING AND MONITORING

- ✓ TEST YOUR INCIDENT RESPONSE PLAN
- ✓ KEEP RUNBOOKS UP TO DATE
- ✓ SEGREGATE ACCOUNTS AND DUTIES
- ✓ REVIEW AND IMPROVE CONTINUOUSLY



**SPEED + PREPARATION
= MINIMAL IMPACT
+ FASTER RECOVERY**

PRODUCTION AWS SECURITY ARCHITECTURE AND FINAL REVIEW


BUILD A SECURE, RESILIENT, AND COMPLIANT AWS ENVIRONMENT AT SCALE
COMPLETE SECURE MULTI-ACCOUNT ARCHITECTURE

CENTRAL SECURITY SERVICES

INCIDENT-RESPONSE WORKFLOW

PRODUCTION SECURITY CHECKLIST

- ✓ Root user secured with MFA
- ✓ Use IAM Identity Center (SSO)
- ✓ Least privilege access enforced
- ✓ SCPs applied at OU level
- ✓ CloudTrail enabled (org trail)
- ✓ Logs stored in Log Archive account
- ✓ GuardDuty enabled in all accounts
- ✓ Security Hub enabled
- ✓ AWS Config enabled with rules
- ✓ Inspector scanning enabled
- ✓ KMS used for all encryption
- ✓ WAF enabled for applications
- ✓ Network uses private subnets
- ✓ VPC endpoints for AWS access
- ✓ Secrets stored and rotated
- ✓ Monitoring and alerts in place
- ✓ Backups encrypted and tested
- ✓ Incident-response plan tested

COMMON SECURITY MISTAKES

- ✗ Using root user for daily tasks
- ✗ Overly permissive IAM policies
- ✗ No MFA on privileged accounts
- ✗ Public S3 buckets and resources
- ✗ No log centralization or retention
- ✗ Ignoring security alerts and findings
- ✗ Hardcoding secrets in code
- ✗ No vulnerability scanning
- ✗ Missing encryption at rest/in transit
- ✗ Flat networks with no segmentation
- ✗ No incident-response plan
- ✗ Not reviewing access regularly


SENIOR ENGINEER RECOMMENDATIONS

- ★ Automate security guardrails with IaC and SCPs.
- ★ Continuously monitor and respond, not just collect logs.
- ★ Follow the principle of least privilege everywhere.
- ★ Test your incident-response plan regularly.
- ★ Keep security simple, repeatable, and measurable.
- ★ Security is not a checklist. It is a culture.


**SECURE TODAY.
PROTECT TOMORROW.**

VERIQTA – ENGINEERING EXCELLENCE



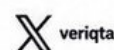
veriqta



veriqta



veriqta



veriqta



veriqta



veriqta